

EEE8089

M2M Technology IoT

Topic 4: Control System over M2M

Dr. Bystrov

School of Engineering
University of Newcastle upon Tyne

Aims/Objectives

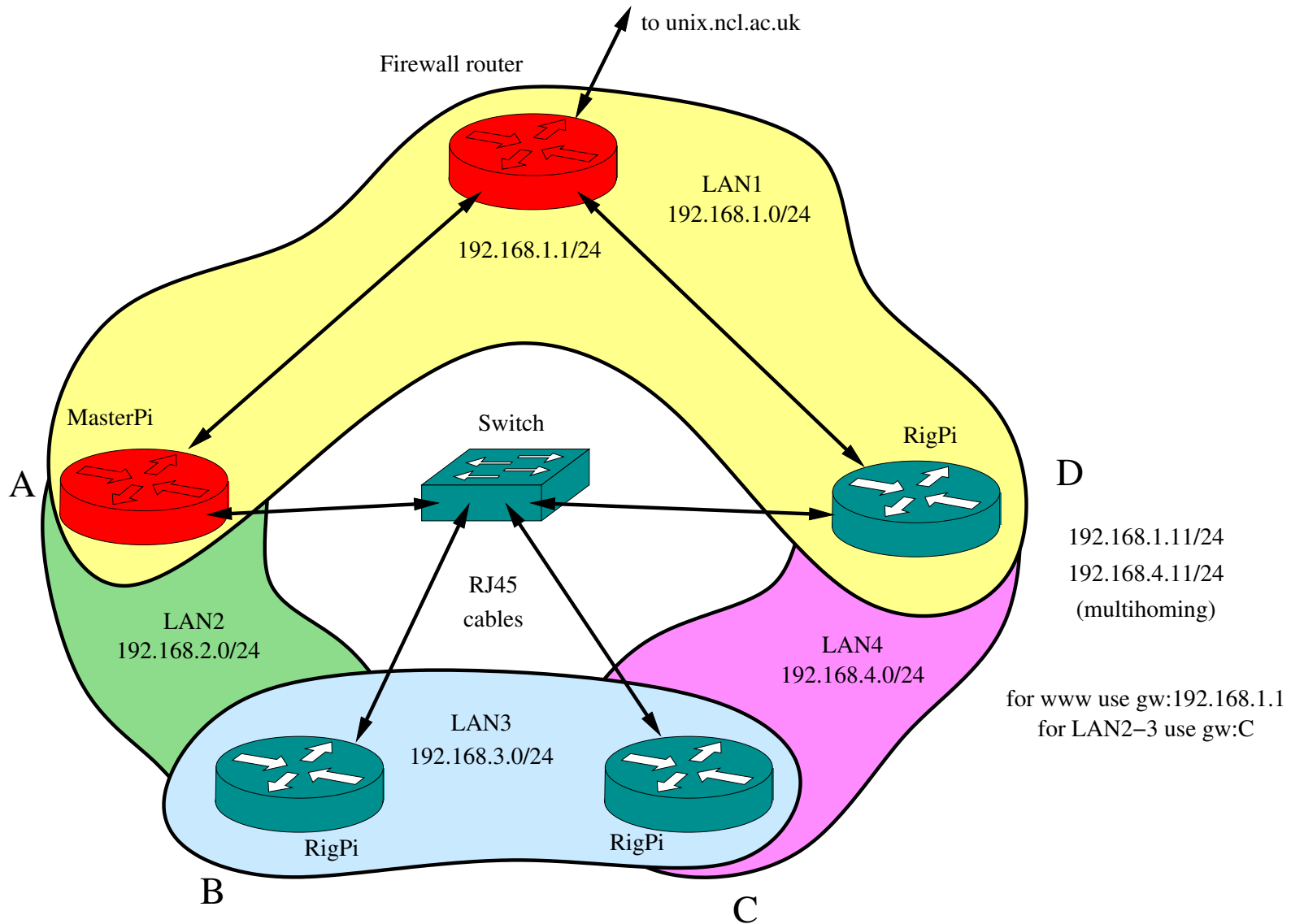
Aim

- M2M architectures for the experiments
- DC motor closed loop control

Objectives

- Connected LANs with routing
- Closed loop DC motor control system
- Raspberry PI interrupts
- Measurement of the motor speed
- Raspberry PI RS232 interface
- Communication to the driver board

Connected LANs for the experiment



Configuration of the LANs

- For different destination networks different gateways are used
- Several (two) LANs may exist on a single interface (multihoming)

Setting the routing tables

- Open the firewall, set as default in our case, e.g. for host C:

```
iptables -v -A FORWARD -s 192.168.4.0/24 -d 192.168.2.0/24 -j ACCEPT
```

- Setup the routing rule, e.g. for host D (gw is host C)

```
route add -net 192.168.2.0 netmask 255.255.255.0 gw 192.168.3.011
```

Multihoming

- Several LANs on the same interface (e.g. RigPi C)
 - e.g. eth0 serves 192.168.1.11/24 and 192.168.4.11/24
- The device (e.g. eth0) can be connected to the same switch with other LANs

/etc/network/interfaces:

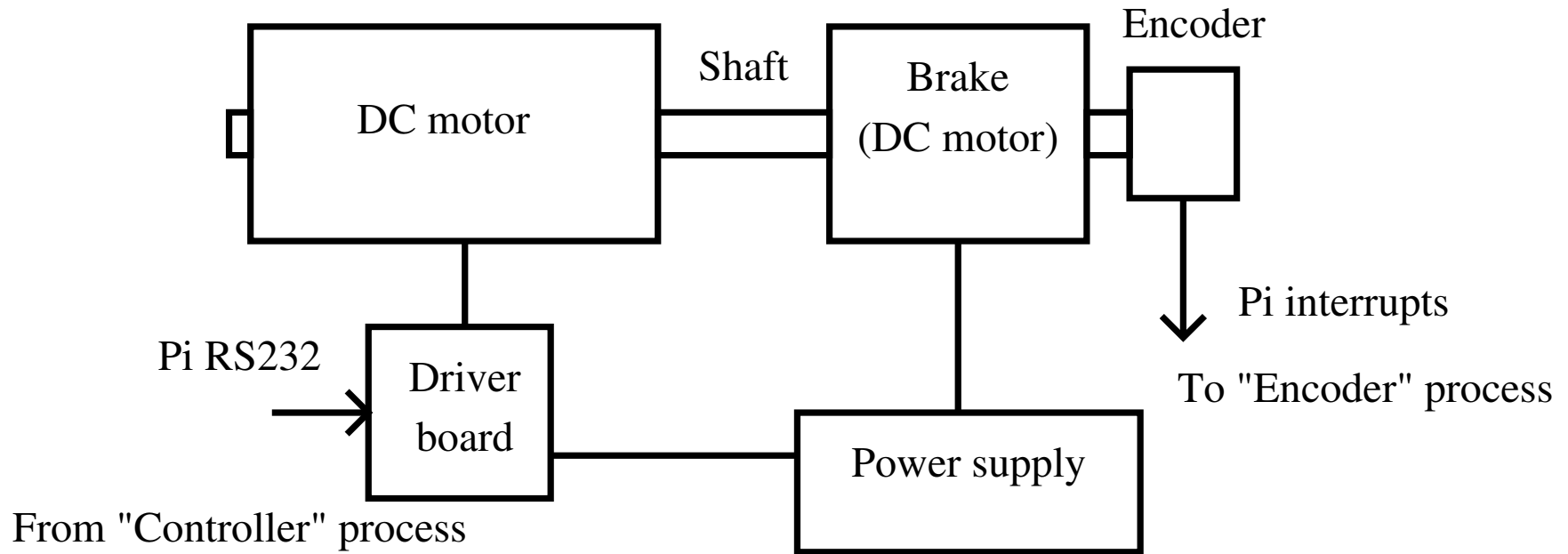
```
auto eth0
allow-hotplug eth0
iface eth0 inet static
    ...address1, etc.

auto eth0:0
allow-hotplug eth0:0
iface eth0:0 inet static
    ...address2, etc.
```

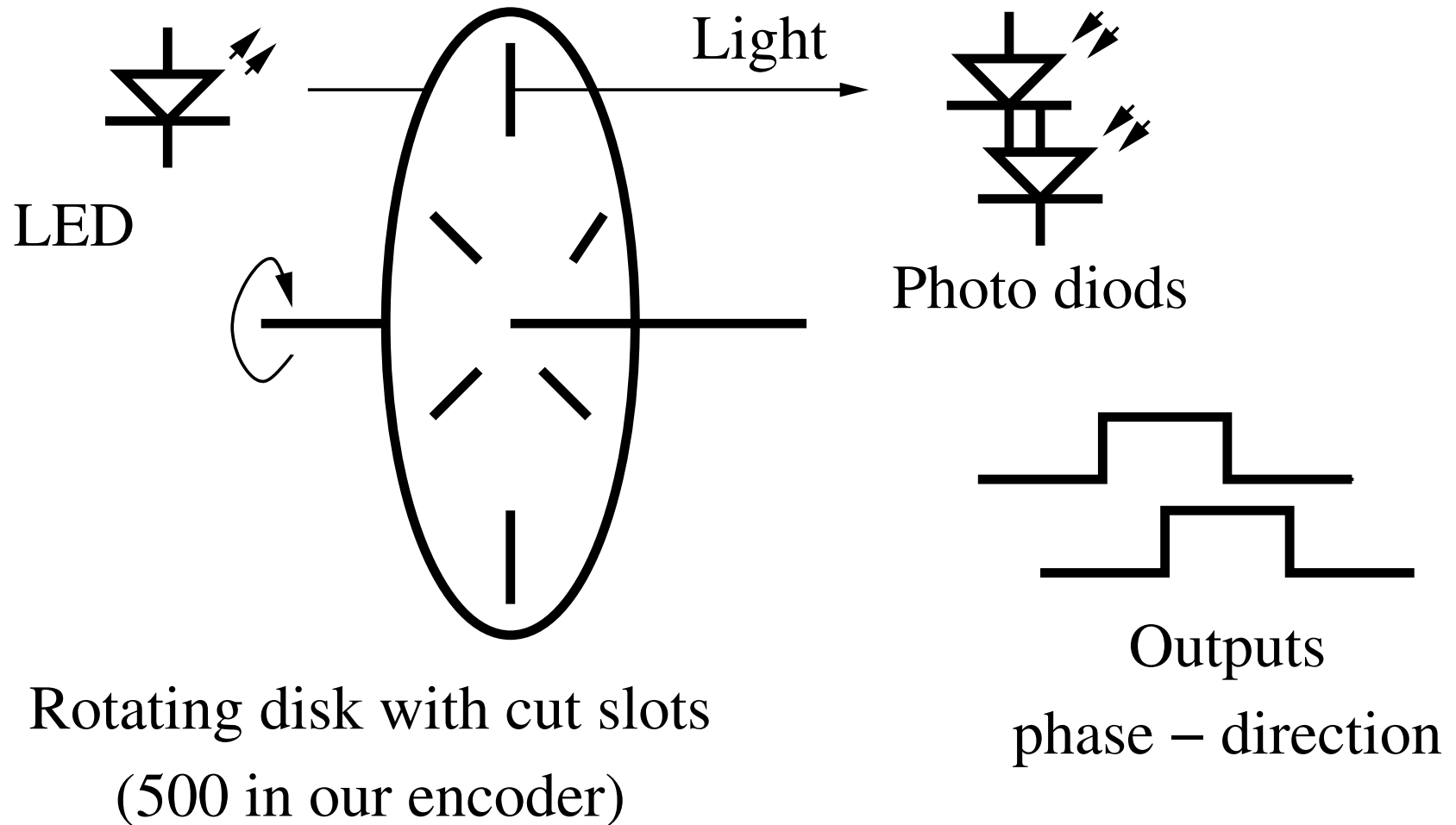
Experiment

- Verify the correctness of packet routing
- Connect the motor driver board and the sensor:
 - to the same device
 - to two different devices with routing through 1, 2, 3 LANs
 - the same with routing through `unix.ncl.ac.uk`
 - observe the impact of the connection latency on stability of the closed loop control system

Control system – the rig



Control system – encoder



Control system – driver board

- RS232 duplex 19200 bps
- Input format: “%d %d\\”
 - motor input: 737 – off, +-500 left-right direction
 - enable/disable 1/0
- /dev/serial0
 - “echo 800 1\\ > /dev/serial0”
 - “cat /dev/serial0”
- RS232 has to be enabled in the system settings of Pi
- Power supply to the board +-20V

Control system – interrupts for encoder

To

configure e.g. IRQ9:

- `echo 9 > /sys/class/gpio/unexport`
- `echo 9 > /sys/class/gpio/export`
- `echo in > /sys/class/gpio/gpio9/direction`
- `echo rising > /sys/class/gpio/gpio9/edge`

To

access the interrupt input:

- use the standard library poll, header `<poll.h>`
- use function `poll()`, which freezes and then is released by the interrupt
- read “man poll”
- example of use is provided on Blackboard

Control system – reading one pin

To

configure e.g. IRQ10:

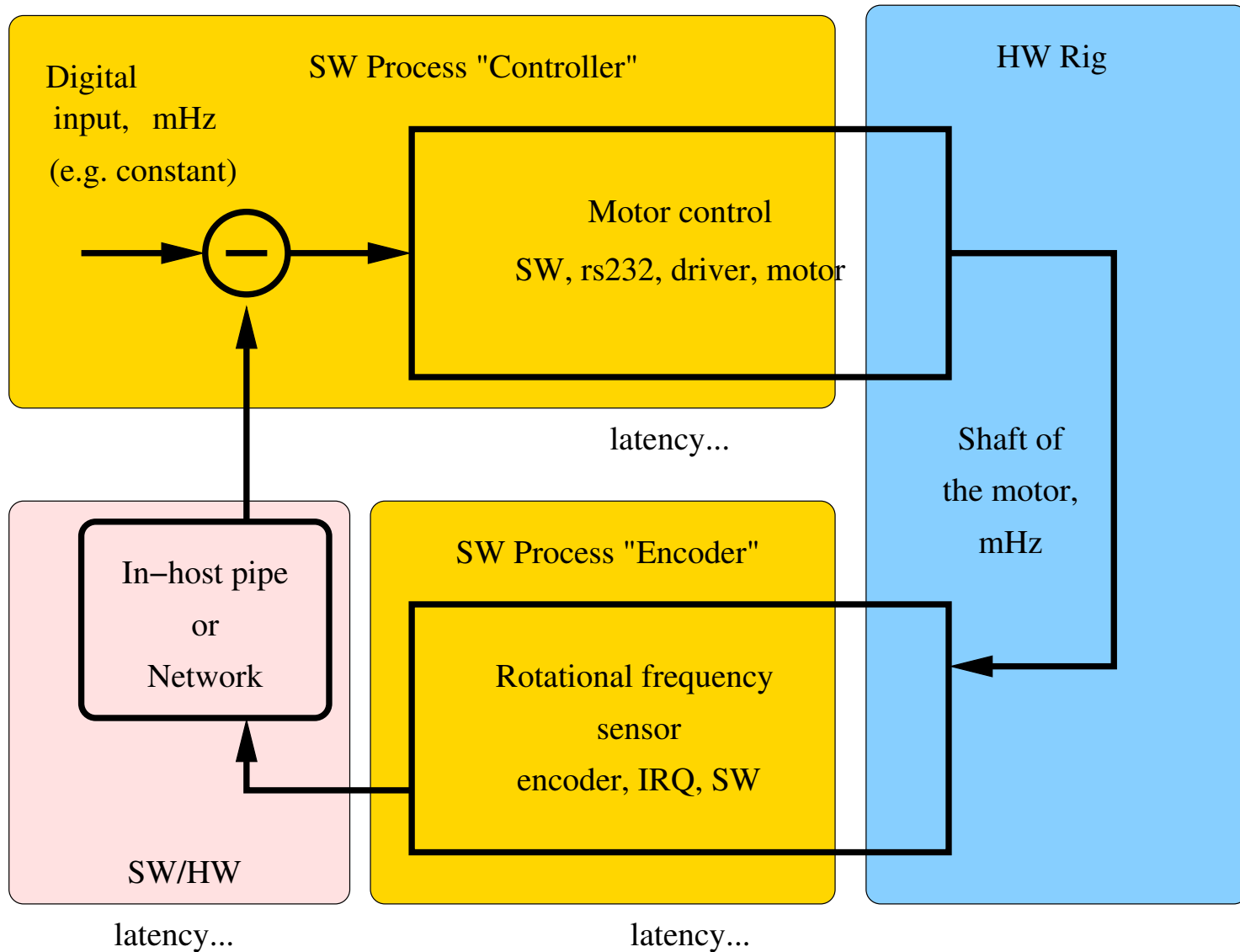
- `echo 10 > /sys/class/gpio/unexport`
- `echo 10 > /sys/class/gpio/export`
- `echo in > /sys/class/gpio/gpio10/direction`
- `echo none > /sys/class/gpio/gpio10/edge`

To

access the GPIO input:

- read the value from `/sys/class/gpio/gpio10/value`
- For example, `cat /sys/class/gpio/gpio10/value`

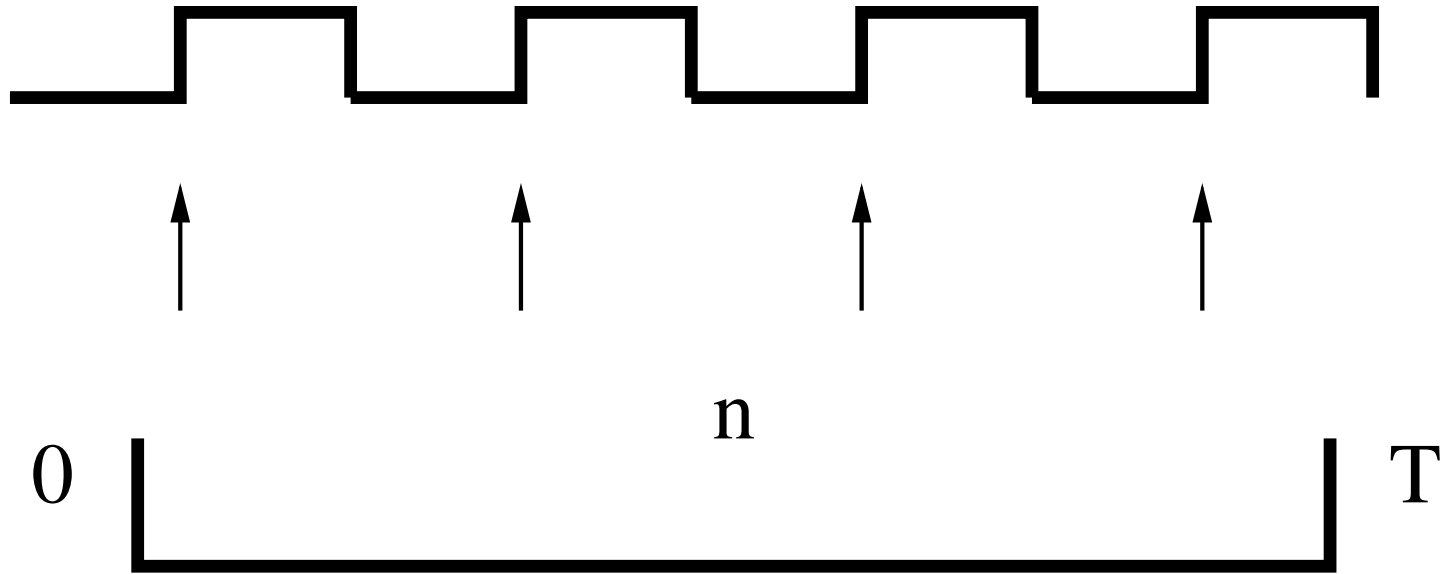
Control system model



Frequency measurement

signal

IRQ9



Timer

start

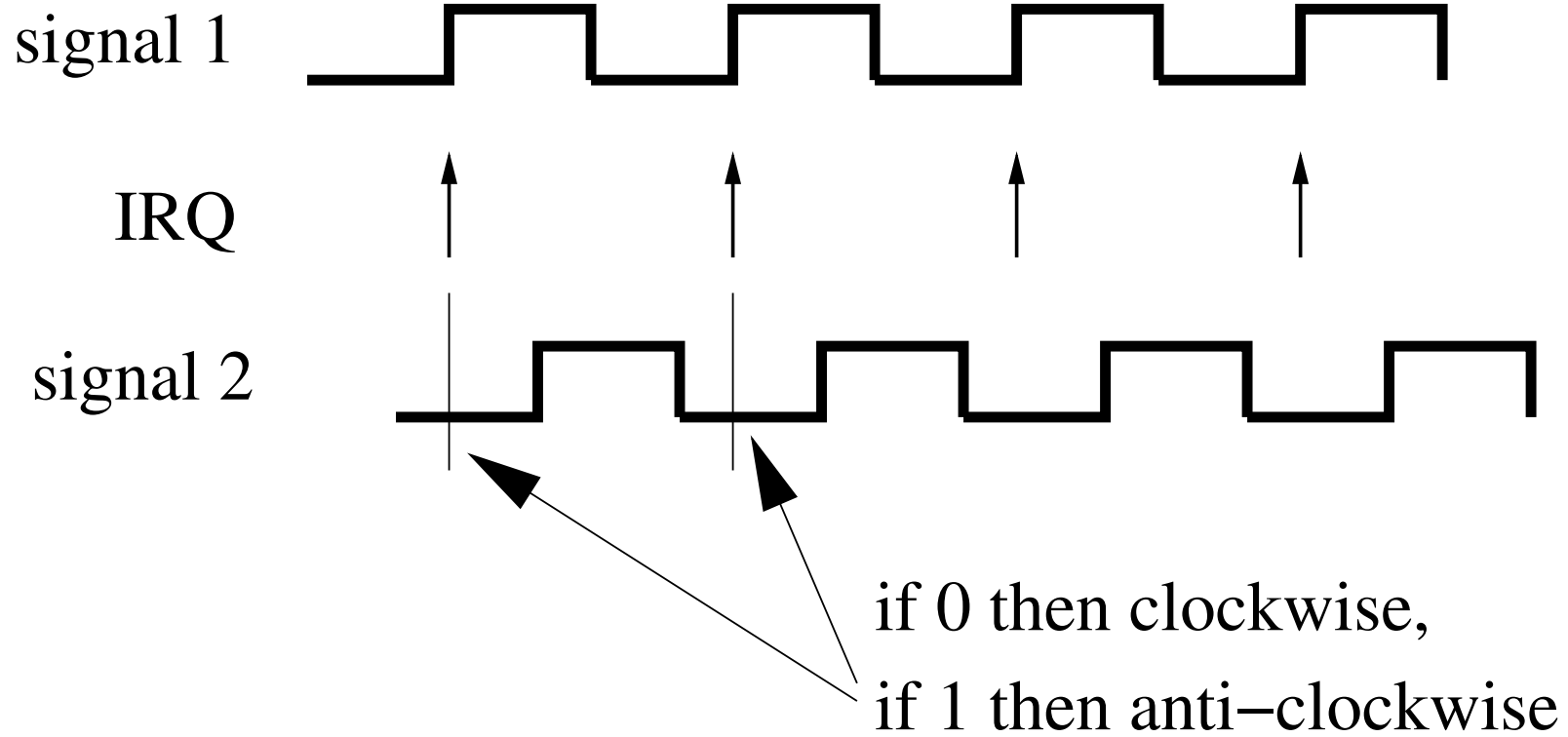
Timer

stop

$$f = n / T_{\text{sample}}$$

- +- 10 rotations per second, 500 pulses per rotation
- T translates into both the accuracy and latency

Direction measurement



System pipeline

- Simple experiment

```
./encoder | ./regulator > /dev/serial0
```

- Over a single SSH connection

```
./encoder | ssh host2 ./regulator ">"  
/dev/serial0
```

- Over M2M which connects the fifo file "ff" in one host to a similar file in the other host

```
host1:      ./encoder > ff
```

```
host2:      cat ff | ./regulator > /dev/serial0
```